

```

//-----
// File name: Exercises.cpp
// Assign ID:
// Due Date: 22/08/24 at 11pm
//
// Purpose: Structures.
//
// Author: Mr. KEO Sopahnit
//-----

// ----- Exercise_1 -----
#include <iostream>
using namespace std;

struct ComplexNumber {
    double real;
    double imag;
    // Constructor
    ComplexNumber(double r = 0, double i = 0) : real(r), imag(i) {}
};

// Addition of two complex numbers
ComplexNumber addition(ComplexNumber num1, ComplexNumber num2) {
    ComplexNumber result;
    result.real = num1.real + num2.real;
    result.imag = num1.imag + num2.imag;
    return result;
}

// Subtraction of two complex numbers
ComplexNumber subtraction(ComplexNumber num1, ComplexNumber num2) {
    ComplexNumber result;
    result.real = num1.real - num2.real;
    result.imag = num1.imag - num2.imag;
    return result;
}

// Multiplication of two complex numbers
//  $(a+bi) * (c+di) = (ac-bd) + (ad+bc)i$ 
ComplexNumber multiplication(ComplexNumber num1, ComplexNumber num2) {
    ComplexNumber result;
    result.real = (num1.real * num2.real) - (num1.imag * num2.imag);
    result.imag = (num1.real * num2.imag) + (num1.imag * num2.real);
    return result;
}

// Division of two complex numbers
//  $(a+bi) / (c+di) = (ac+bd) / (c^2+d^2) + (bc-ad)i / (c^2+d^2)$ 
ComplexNumber division(ComplexNumber num1, ComplexNumber num2) {
    ComplexNumber result;
    double denominator = num2.real * num2.real + num2.imag * num2.imag;
    if (denominator == 0) {
        throw invalid_argument("Cannot divide by zero complex number");
    }
}

```

```

    result.real = (num1.real * num2.real + num1.imag * num2.imag) /
denominator;
    result.imag = (num1.imag * num2.real - num1.real * num2.imag) /
denominator;
    return result;
}

// Function to cout the operation in the print (a+bi) + (c+di)
void print(ComplexNumber num1, ComplexNumber num2, string operate) {
    cout << "(" << num1.real << " + " << num1.imag << "i)" << operate << "("
        << num2.real << " + " << num2.imag << "i)" << " = ";
}

// Function to print the result of a complex number operation
void results(ComplexNumber result) {
    if (result.imag >= 0) {
        cout << result.real << " + " << result.imag << "i" << endl;
    } else {
        cout << result.real << " - " << -result.imag << "i" << endl;
    }
}

int main() {
    ComplexNumber z1(1, 4); // 3 + 4i
    ComplexNumber z2(4, 7); // 1 + 7i

    // Addition
    print(z1, z2, " + ");
    results(addition(z1, z2));

    // Subtraction
    print(z1, z2, " - ");
    results(subtraction(z1, z2));

    // Multiplication
    print(z1, z2, " * ");
    results(multiplication(z1, z2));

    // Division
    print(z1, z2, " / ");
    results(division(z1, z2));

    return 0;
}

```

```

// ----- Exercise_2 -----
#include <iostream>
using namespace std;

struct Car {
    // Member variables
    double length;
    double clearance;
    double engineSize;
    double enginePower;
    double wheelDiameter;
    string color;
    string gearboxType;
    // Constructor with member initialization
    Car(double len, double clr, double engSize, double engPower, double
wheelDia,
        string clrCar, string gbxType)
        : length(len), clearance(clr), engineSize(engSize),
enginePower(engPower),
        wheelDiameter(wheelDia), color(clrCar), gearboxType(gbxType) {}
};

void display(Car car);
int main() {
    // Create a Car object with specified attributes
    Car myCar(4.5, 20.0, 2.0, 150, 18, "Red", "Automatic");

    // Display car details
    display(myCar);

    return 0;
}

// Function to display car details
void display(Car car) {
    cout << "\tCar Specifications:" << endl;
    cout << "Length: " << car.length << " meters" << endl;
    cout << "Clearance (Ride Height): " << car.clearance << " cm" << endl;
    cout << "Engine Size: " << car.engineSize << " liters" << endl;
    cout << "Engine Power: " << car.enginePower << " HP" << endl;
    cout << "Wheel Diameter: " << car.wheelDiameter << " inches" << endl;
    cout << "Color: " << car.color << endl;
    cout << "Gearbox Type: " << car.gearboxType << endl;
}

// ----- Exercise_3 -----
#include <algorithm> // For std::sort
#include <iomanip>
#include <iostream>
#include <string>
using namespace std;

// Structure representing a book
struct Book {
    string title;

```

```

    string author;
    string publisher;
    string genre;
};

// Function
void editBook(Book &book);
void printBooks(const Book books[], int size);
void searchByAuthor(const Book books[], int size, const string &author);
void searchByTitle(const Book books[], int size, const string &title);
void sortByTitle(Book books[], int size);
void sortByAuthor(Book books[], int size);
void sortByPublisher(Book books[], int size);

int main() {
    const int SIZE = 5;
    Book library[SIZE] = {
        {"The Catcher in the Rye", "J.D. Salinger", "Little, Brown and
Company",
        "Fiction"},
        {"To Kill a Mockingbird", "Harper Lee", "J.B. Lippincott & Co.",
        "Fiction"},
        {"1984", "George Orwell", "Secker & Warburg", "Dystopian"},
        {"Moby-Dick", "Herman Melville", "Harper & Brothers", "Adventure"},
        {"The Great Gatsby", "F. Scott Fitzgerald", "Charles Scribner's
Sons",
        "Fiction"},
    };

    int choice;
    do {
        cout << "\n==== Library Menu =====\n";
        cout << "1. Edit a book\n";
        cout << "2. Print all books\n";
        cout << "3. Search books by author\n";
        cout << "4. Search books by title\n";
        cout << "5. Sort books by title\n";
        cout << "6. Sort books by author\n";
        cout << "7. Sort books by publisher\n";
        cout << "8. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        cin.ignore(); // To ignore newline character after input
        switch (choice) {
            case 1: {
                int index;
                cout << "Enter book number (1-10) to edit: ";
                cin >> index;
                if (index >= 1 && index <= SIZE) {
                    editBook(library[index - 1]);
                } else {
                    cout << "Invalid book number!" << endl;
                }
            }

```

```

        break;
    }
    case 2:
        printBooks(library, SIZE);
        break;
    case 3: {
        string author;
        cout << "Enter author name: ";
        getline(cin, author);
        searchByAuthor(library, SIZE, author);
        break;
    }
    case 4: {
        string title;
        cout << "Enter book title: ";
        getline(cin, title);
        searchByTitle(library, SIZE, title);
        break;
    }
    case 5:
        sortByTitle(library, SIZE);
        cout << "Books sorted by title." << endl;
        break;
    case 6:
        sortByAuthor(library, SIZE);
        cout << "Books sorted by author." << endl;
        break;
    case 7:
        sortByPublisher(library, SIZE);
        cout << "Books sorted by publisher." << endl;
        break;
    case 8:
        cout << "Exiting program..." << endl;
        break;
    default:
        cout << "Invalid choice! Please try again." << endl;
    }
} while (choice != 8);

return 0;
}

// Function to edit a book's details
void editBook(Book &book) {
    cout << "Editing book: " << book.title << endl;
    cout << "Enter new title: ";
    getline(cin, book.title);
    cout << "Enter new author: ";
    getline(cin, book.author);
    cout << "Enter new publisher: ";
    getline(cin, book.publisher);
    cout << "Enter new genre: ";
    getline(cin, book.genre);
}

```

```

// Function to print all books in the library
void printBooks(const Book books[], int size) {
    cout << "\n===== All Books =====\n";
    cout << setw(10) << left << "Book No." << setw(30) << left << "Title"
        << setw(20) << left << "Author" << setw(20) << left << "Publisher"
        << setw(20) << left << "Genre" << endl;
    cout << string(95, '-') << endl;

    for (int i = 0; i < size; ++i) {
        cout << setw(10) << left << i + 1 << setw(30) << left <<
books[i].title
        << setw(20) << left << books[i].author << setw(20) << left
        << books[i].publisher << setw(20) << left << books[i].genre <<
endl;
    }
}

// Function to search for books by a specific author
void searchByAuthor(const Book books[], int size, const string &author) {
    cout << "\nBooks by " << author << ":\n";
    bool found = false;
    for (int i = 0; i < size; ++i) {
        if (books[i].author == author) {
            cout << i + 1 << ". Title: " << books[i].title << endl;
            found = true;
        }
    }
    if (!found) {
        cout << "No books found by that author." << endl;
    }
}

// Function to search for books by title
void searchByTitle(const Book books[], int size, const string &title) {
    cout << "\nSearching for book: " << title << endl;
    bool found = false;
    for (int i = 0; i < size; ++i) {
        if (books[i].title == title) {
            cout << "Found: " << books[i].title << " by " << books[i].author <<
endl;
            found = true;
        }
    }
    if (!found) {
        cout << "No books found with that title." << endl;
    }
}

// Function to sort books by title
void swapBooks(Book &a, Book &b) {
    Book temp = a;
    a = b;
    b = temp;
}

```

```

}

// Bubble sort function to sort books by title
void sortByTitle(Book books[], int size) {
    for (int i = 0; i < size - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < size - i - 1; j++) {
            if (books[j].title > books[j + 1].title) {
                swapBooks(books[j], books[j + 1]);
                swapped = true;
            }
        }
        // If no two elements were swapped, the array is already sorted
        if (!swapped) {
            break;
        }
    }
    cout << "Sorted by title in ascending order successfully!" << endl;
}

// Bubble sort function to sort books by title
void sortByAuthor(Book books[], int size) {
    for (int i = 0; i < size - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < size - i - 1; j++) {
            if (books[j].author > books[j + 1].author) {
                swapBooks(books[j], books[j + 1]);
                swapped = true;
            }
        }
        // If no two elements were swapped, the array is already sorted
        if (!swapped) {
            break;
        }
    }
    cout << "Sorted by title in ascending order successfully!" << endl;
}

// Bubble sort function to sort books by title
void sortByPublisher(Book books[], int size) {
    for (int i = 0; i < size - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < size - i - 1; j++) {
            if (books[j].publisher > books[j + 1].publisher) {
                swapBooks(books[j], books[j + 1]);
                swapped = true;
            }
        }
        // If no two elements were swapped, the array is already sorted
        if (!swapped) {
            break;
        }
    }
    cout << "Sorted by title in ascending order successfully!" << endl;
}

```

```

-----Exercise_3-----
#include <iomanip>
#include <iostream>
using namespace std;

// Union for car number (either a five-digit number or a word up to 8
// characters)
union CarNumber {
    int number;    // Five-digit number
    char word[9]; // Word up to 8 characters + null terminator
};

// Enum to track which type of number is stored in the union
enum NumberType { NUMERIC, ALPHANUMERIC };
enum Option {EXIT,ADD,EDIT,PRINT,SEARCH};

// Structure for Car
struct Car {
    string color;
    string model;
    CarNumber number;
    NumberType numType;
};

template <typename T> void input(T inPut, string text) {
    cout << text;
    cin >> inPut;
}

// Function to fill car details
void fillCar(Car &car) {
    int choice;
    input(car.color, "Enter car color: ");
    input(car.model, "Enter car model: ");
    input(choice, "Is the car number a \n1) Numeric value \n2) Alphanumeric
word "
           "\nEnter 1 or 2: ");

    if (choice == 1) {
        car.numType = NUMERIC;
        cout << "Enter the car's numeric number (five-digit): ";
        cin >> car.number.number;
    } else {
        car.numType = ALPHANUMERIC;
        cout << "Enter the car's alphanumeric number (up to 8 characters): ";
        cin >> car.number.word;
    }
}

// Function to print car details
void printCar(const Car &car) {
    cout << "\nCar Details: \n";
    cout << "Color: " << car.color << "\n";
    cout << "Model: " << car.model << "\n";
    if (car.numType == NUMERIC) {

```

```

        cout << "Number: " << car.number.number << "\n";
    } else {
        cout << "Number: " << car.number.word << "\n";
    }
}

// Function to edit car details
void editCar(Car &car) { fillCar(car); }

// Function to print all cars in the array
void printAllCars(const Car cars[], int size) {
    cout << "\n===== Car List =====\n";
    for (int i = 0; i < size; ++i) {
        cout << "Car " << i + 1 << ":\n";
        printCar(cars[i]);
        cout << "-----\n";
    }
}

// Function to search for a car by number
void searchByNumber(const Car cars[], int size) {
    int choice;
    input(choice, "Do you want to search by \n1) Numeric number or \n2)
Alphanumeric ");
    "number? \n Enter 1 or 2: ");
    if (choice == 1) {
        int searchNum;
        input(searchNum, "Enter the numeric number: ");
        for (int i = 0; i < size; ++i) {
            if (cars[i].numType == NUMERIC && cars[i].number.number ==
searchNum) {
                cout << "Car found:\n";
                printCar(cars[i]);
                return;
            }
        }
        cout << "Car with numeric number " << searchNum << " not found.\n";
    } else {
        char searchWord[9];
        input(searchWord, "Enter the alphanumeric number: ");
        for (int i = 0; i < size; ++i) {
            if (cars[i].numType == ALPHANUMERIC &&
(cars[i].number.word, searchWord) == 0) {
                cout << "Car found:\n";
                printCar(cars[i]);
                return;
            }
        }
        cout << "Car with alphanumeric number " << searchWord << " not
found.\n";
    }
}

int main() {

```

```

const int SIZE = 10;
Car cars[SIZE]; // Array of 10 cars

int option;
int carIndex;

// Fill 3 cars initially for testing purposes
for (int i = 0; i < 3; i++) {
    cout << "\nEnter details for Car " << i + 1 << ":\n";
    fillCar(cars[i]);
}

while (true) {
    cout << "\n===== Menu =====\n";
    cout << "1. Add/Fill Car\n";
    cout << "2. Edit Car\n";
    cout << "3. Print All Cars\n";
    cout << "4. Search Car by Number\n";
    cout << "5. Exit\n";
    cout << "Enter option: ";
    cin >> option;

    switch (option) {
    case ADD:
        if (carIndex < SIZE) {
            cout << "Enter details for Car " << carIndex + 1 << ":\n";
            fillCar(cars[carIndex++]);
        } else {
            cout << "Car array is full!\n";
        }
        break;

    case EDIT:
        cout << "Enter the car index to edit (1 to " << SIZE << "): ";
        cin >> carIndex;
        if (carIndex >= 1 && carIndex <= SIZE) {
            editCar(cars[carIndex - 1]);
        } else {
            cout << "Invalid car index.\n";
        }
        break;

    case PRINT:
        printAllCars(cars, carIndex);
        break;

    case SEARCH:
        searchByNumber(cars, carIndex);
        break;

    case EXIT:
        return 0;

    default:

```

```
        cout << "Invalid option.\n";
    }
}
return 1;
}
```

```

-----Exercise_4-----
#include <iomanip>
#include <iostream>
using namespace std;

// Union for car number (either a five-digit number or a word up to 8
// characters)
union CarNumber {
    int number;    // Five-digit number
    char word[9]; // Word up to 8 characters + null terminator
};

// Enum to track which type of number is stored in the union
enum NumberType { NUMERIC, ALPHANUMERIC };
enum Option {EXIT,ADD,EDIT,PRINT,SEARCH};

// Structure for Car
struct Car {
    string color;
    string model;
    CarNumber number;
    NumberType numType;
};

template <typename T> void input(T inPut, string text) {
    cout << text;
    cin >> inPut;
}

// Function to fill car details
void fillCar(Car &car) {
    int choice;
    input(car.color, "Enter car color: ");
    input(car.model, "Enter car model: ");
    input(choice, "Is the car number a \n1) Numeric value \n2) Alphanumeric
word "
           "\nEnter 1 or 2: ");

    if (choice == 1) {
        car.numType = NUMERIC;
        cout << "Enter the car's numeric number (five-digit): ";
        cin >> car.number.number;
    } else {
        car.numType = ALPHANUMERIC;
        cout << "Enter the car's alphanumeric number (up to 8 characters): ";
        cin >> car.number.word;
    }
}

// Function to print car details
void printCar(const Car &car) {
    cout << "\nCar Details: \n";
    cout << "Color: " << car.color << "\n";
    cout << "Model: " << car.model << "\n";
    if (car.numType == NUMERIC) {

```

```

        cout << "Number: " << car.number.number << "\n";
    } else {
        cout << "Number: " << car.number.word << "\n";
    }
}

// Function to edit car details
void editCar(Car &car) { fillCar(car); }

// Function to print all cars in the array
void printAllCars(const Car cars[], int size) {
    cout << "\n===== Car List =====\n";
    for (int i = 0; i < size; ++i) {
        cout << "Car " << i + 1 << ":\n";
        printCar(cars[i]);
        cout << "-----\n";
    }
}

// Function to search for a car by number
void searchByNumber(const Car cars[], int size) {
    int choice;
    input(choice, "Do you want to search by \n1) Numeric number or \n2)
Alphanumeric ");
    "number? \n Enter 1 or 2: ");
    if (choice == 1) {
        int searchNum;
        input(searchNum, "Enter the numeric number: ");
        for (int i = 0; i < size; ++i) {
            if (cars[i].numType == NUMERIC && cars[i].number.number ==
searchNum) {
                cout << "Car found:\n";
                printCar(cars[i]);
                return;
            }
        }
        cout << "Car with numeric number " << searchNum << " not found.\n";
    } else {
        char searchWord[9];
        input(searchWord, "Enter the alphanumeric number: ");
        for (int i = 0; i < size; ++i) {
            if (cars[i].numType == ALPHANUMERIC &&
(cars[i].number.word, searchWord) == 0) {
                cout << "Car found:\n";
                printCar(cars[i]);
                return;
            }
        }
        cout << "Car with alphanumeric number " << searchWord << " not
found.\n";
    }
}

int main() {

```

```

const int SIZE = 10;
Car cars[SIZE]; // Array of 10 cars

int option;
int carIndex;

// Fill 3 cars initially for testing purposes
for (int i = 0; i < 3; i++) {
    cout << "\nEnter details for Car " << i + 1 << ":\n";
    fillCar(cars[i]);
}

while (true) {
    cout << "\n===== Menu =====\n";
    cout << "1. Add/Fill Car\n";
    cout << "2. Edit Car\n";
    cout << "3. Print All Cars\n";
    cout << "4. Search Car by Number\n";
    cout << "5. Exit\n";
    cout << "Enter option: ";
    cin >> option;

    switch (option) {
    case ADD:
        if (carIndex < SIZE) {
            cout << "Enter details for Car " << carIndex + 1 << ":\n";
            fillCar(cars[carIndex++]);
        } else {
            cout << "Car array is full!\n";
        }
        break;

    case EDIT:
        cout << "Enter the car index to edit (1 to " << SIZE << "): ";
        cin >> carIndex;
        if (carIndex >= 1 && carIndex <= SIZE) {
            editCar(cars[carIndex - 1]);
        } else {
            cout << "Invalid car index.\n";
        }
        break;

    case PRINT:
        printAllCars(cars, carIndex);
        break;

    case SEARCH:
        searchByNumber(cars, carIndex);
        break;

    case EXIT:
        return 0;

    default:

```

```
        cout << "Invalid option.\n";
    }
}
return 1;
}
```